

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted By

PIYUSH KUMAR (1BF24CS216)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by Piyush Kumar(**1BF24CS216**), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - (**23CS4PCADA**) work prescribed for the said degree.

Dr. Jyothi S Nayak
Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4
2	Implement Johnson Trotter algorithm to generate permutations.	7
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	9
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	12
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	15
6	Implement 0/1 Knapsack problem using dynamic programming. (5 Marks)	22
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	24
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	26 29
9	Implement Fractional Knapsack using Greedy technique.	33
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	36
11	Implement "N-Queens Problem" using Backtracking.	39

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

C code

```
#include<stdio.h> #include<conio.h> #define max 5 int main()
{ int i,j,k=0; int count=0; int n; int topo[max]; int
adj[max][max]; int indegree[max];
printf("\nEnter the number of nodes in the
graph:"); scanf("%d",&n); printf("\nEnter the
adjacency matrix of the graph:\n");
for(i=0;i<n;i++)
{
for(j=0;j<n;j++)
scanf("%d",&adj[i][j]);
}
for(j=0;j<n;j++)
{
indegree[j]=0;
for(i=0;i<n;i++)
{
if(adj[i][j]==1)
indegree[j]++;
}
}
while(count<n)
```

```

{
int found=0;
for(i=0;i<n;i++)
{
if(indegree[i]==0)
{
        topo[count++]=i;
found=1; indegree[i]=-1;
for(j=0;j<n;j++)
{
if(adj[i][j]
==1)
{ adj[i][j]=0;
        indegre
        e[j]--;
}}}}
if(!foun
d)
{
        printf("\nThe graph has a cycle no topologicaal order \n");
return 0;}}
printf("\nThe topological order is :\n");
for(i=0;i<n;i++)
{
        printf("\t%d",topo[i]);
}

```

```
printf("\n");
```

```
return 0;}
```

OUTPUT :

```
Enter the number of nodes in the graph:4
Enter the adjacency matrix of the graph:
0 1 1 0
0 0 1 1
0 0 0 1
0 0 0 0

The topological order is :
    0      1      2      3

Process returned 0 (0x0)   execution time : 41.471 s
Press any key to continue.
```

```
Enter the number of nodes in the graph:3
```

```
Enter the adjacency matrix of the graph:
```

```
0 1 0
```

```
0 0 1
```

```
1 0 0
```

```
The graph has a cycle no topologicaal order
```

```
Process returned 0 (0x0)   execution time : 24.872 s
```

```
Press any key to continue.
```

Lab program 2

Implement Johnson Trotter algorithm to generate permutations.

C code :

```
#include <stdio.h>

int a[10], dir[10];

void print(int n) {
    for(int i = 0; i < n;
        i++)
        printf("%d ", a[i]);
        printf("\n");
    }

int mobile(int n) {
    int m = 0, pos = -1;

    for(int i = 0; i < n; i++) {
        int j = i + dir[i];

        if(j >= 0 && j < n && a[i] > a[j] && a[i]
            > m) {          m = a[i];          pos = i;
        }
    }
    return
    pos;
}

int
main() {
    int n;
    printf("Enter a value :");
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        a[i] = i +
        1;          dir[i] =
        -1;
    }

    while(1) {
        print(n);

        int m = mobile(n);
```



```

        if(m == -1) break;
        int j = m + dir[m];
int t = a[m];      a[m] = a[j];      a[j] = t;

        t = dir[m];
dir[m] = dir[j];
dir[j] = t;

        m = j;

        for(int i = 0; i <
n; i++)      if(a[i]
> a[m])
dir[i] *= -1;
    }

    return 0;
}

```

OUTPUT :

```

Enter a value :3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Process returned 0 (0x0)   execution time : 57.245 s
Press any key to continue.

```

```

Enter a value :4
1 2 3 4
1 2 4 3
1 4 2 3
4 1 2 3
4 1 3 2
1 4 3 2
1 3 4 2
1 3 2 4
3 1 2 4
3 1 4 2
3 4 1 2
4 3 1 2
4 3 2 1
3 4 2 1
3 2 4 1
3 2 1 4
2 3 1 4
2 3 4 1
2 4 3 1
4 2 3 1
4 2 1 3
2 4 1 3
2 1 4 3
2 1 3 4

Process returned 0 (0x0)   execution time : 11.145 s
Press any key to continue.

```

Lab program 3

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

C code :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int a[], int l, int m, int
r)
{
    int i = l, j = m + 1, k = 0;
    int temp[r - l + 1];

    while(i <= m && j <= r)
    {
        if(a[i] < a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while(i <= m)
        temp[k++] = a[i++];

    while(j <= r)
        temp[k++] = a[j++];

    for(i = l, k = 0; i <= r; i++, k++)
        a[i] = temp[k];
}

void mergeSort(int a[], int l, int r)
{
    if(l < r)
    {
        int m = (l + r) / 2;

        mergeSort(a, l, m);
        mergeSort(a, m + 1, r);
```

```

        merge(a, l, m, r);
    }
}

int main()
{
    int n;

    printf("Enter number of
elements: ");
    scanf("%d", &n);

    int a[n];

    srand(time(NULL));

    for(int i = 0; i < n; i++)
        a[i] = rand() % 10000;

    clock_t start, end;

    start = clock();

    for(int i=0;i<1000;i++)
    {
        mergeSort(a,0,n-1);
    }

    end = clock();

    double time_taken =
        ((double)(end - start)) /
CLOCKS_PER_SEC;

    printf("\nFirst 20 Sorted
Elements:\n");

    for(int i = 0; i < n && i < 20;
i++)
        printf("%d ", a[i]);

    printf("\n\nTime Taken = %.3f
milliseconds\n",
        time_taken * 1000);

```

```
return 0;}
```

OUTPUT:

```
Enter number of elements: 100

First 20 Sorted Elements:
14 58 70 100 154 375 389 434 482 496 508 910 924 952 1061 1102 1144 1154 1278 1299

Time Taken = 5.000 milliseconds

Process returned 0 (0x0)   execution time : 8.554 s
Press any key to continue.
|
```

```
Enter number of elements: 500

First 20 Sorted Elements:
8 10 39 44 45 60 62 122 125 136 144 146 159 166 170 180 184 199 214 222

Time Taken = 35.000 milliseconds

Process returned 0 (0x0)   execution time : 8.821 s
Press any key to continue.
|
```

```
Enter number of elements: 1000

First 20 Sorted Elements:
4 5 12 15 43 60 66 78 87 96 99 105 110 114 118 124 126 128 128 129

Time Taken = 74.000 milliseconds

Process returned 0 (0x0)   execution time : 7.523 s
Press any key to continue.
|
```

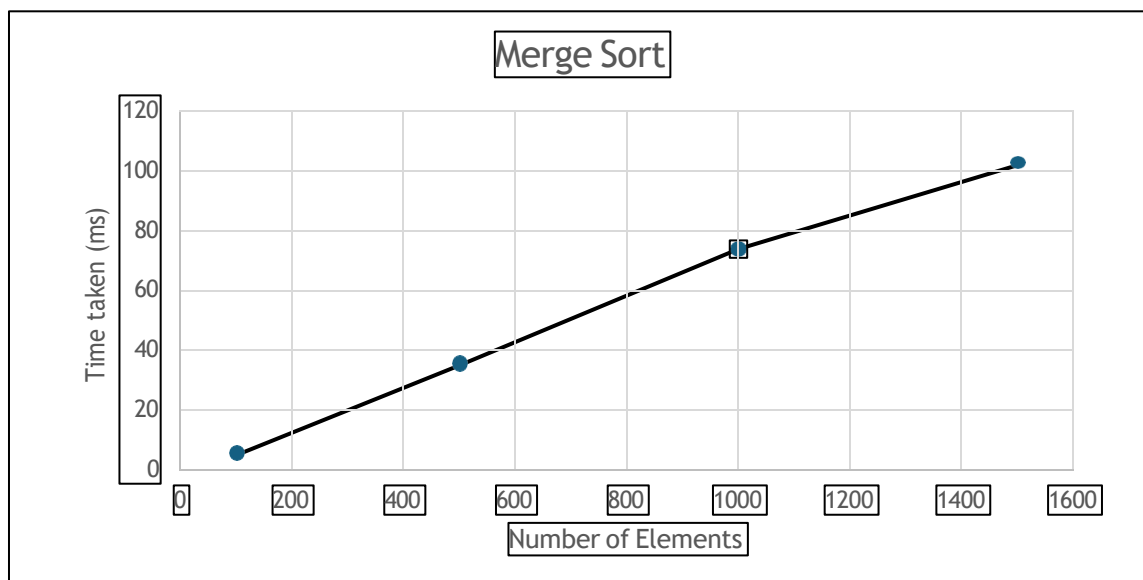
```
Enter number of elements: 1500

First 20 Sorted Elements:
1 5 16 22 25 30 32 39 40 42 43 49 51 57 67 75 86 90 98 99

Time Taken = 102.000 milliseconds

Process returned 0 (0x0)   execution time : 6.314 s
Press any key to continue.
|
```

GRAPH :



Lab program 4

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

C code :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int partition(int a[], int low, int high)
{
    int pivot = a[high];
    int i = low - 1;
    int temp;

    for(int j = low; j < high; j++)
    {
        if(a[j] < pivot)
        {
            i++;

            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }

    temp = a[i + 1];
    a[i + 1] = a[high];
    a[high] = temp;

    return i + 1;
}

void quickSort(int a[], int low, int high)
{
    if(low < high)
    {
        int pi = partition(a, low, high);

        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}
```

```

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int a[n];

    srand(time(NULL));

    for(int i = 0; i < n; i++)
    {
        a[i] = rand() % 10000;
    }

    clock_t start, end;

    start = clock();

    for(int i=0;i<1000;i++)
    {
        quickSort(a,0,n-1);
    }

    end = clock();

    double time_taken =
        ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\nFirst 20 Sorted Elements:\n");

    for(int i = 0; i < n && i < 20; i++)
        printf("%d ", a[i]);

    printf("\n\nTime Taken = %.6f milliseconds\n",
        time_taken * 1000);

    return 0;
}

```

OUTPUT:

```
Enter number of elements: 100

First 20 Sorted Elements:
15 154 231 261 314 320 362 492 542 577 713 724 808 913 934 1031 1134 1168 1211 1221

Time Taken = 20.000000 milliseconds

Process returned 0 (0x0)   execution time : 9.115 s
Press any key to continue.
|
```

```
Enter number of elements: 500

First 20 Sorted Elements:
26 112 113 119 136 151 161 184 203 221 223 227 234 235 249 271 288 293 303 322

Time Taken = 420.000000 milliseconds

Process returned 0 (0x0)   execution time : 9.283 s
Press any key to continue.
|
```

```
Enter number of elements: 1000

First 20 Sorted Elements:
13 17 23 26 39 53 77 77 79 90 96 99 100 105 110 111 118 124 132 140

Time Taken = 1514.000000 milliseconds

Process returned 0 (0x0)   execution time : 10.612 s
Press any key to continue.
|
```



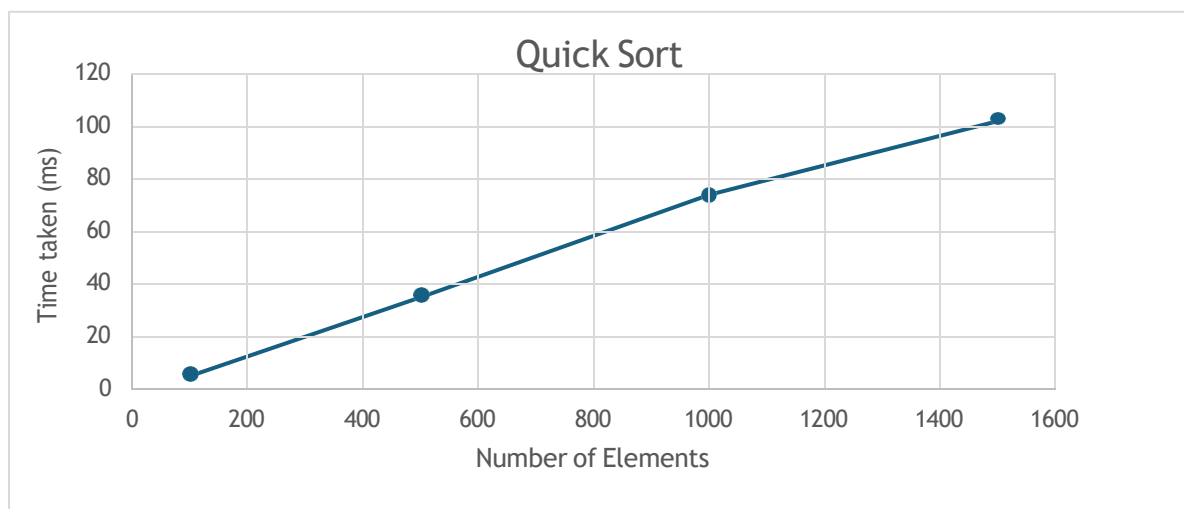
```
Enter number of elements: 1500

First 20 Sorted Elements:
1 11 12 13 14 19 21 23 34 41 43 46 48 60 63 71 76 81 83 99

Time Taken = 3252.000000 milliseconds

Process returned 0 (0x0)   execution time : 10.615 s
Press any key to continue.
|
```

GRAPH :



Lab program 5

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

C code :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void heapify(int a[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

    if(left < n && a[left] > a[largest])
        largest = left;

    if(right < n && a[right] > a[largest])
        largest = right;

    if(largest != i)
    {
        temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;

        heapify(a, n, largest);
    }
}

void heapSort(int a[], int n)
{
    for(int i = n/2 - 1; i >= 0; i--)
        heapify(a, n, i);

    for(int i = n - 1; i > 0; i--)
    {
        int temp = a[0];
        a[0] = a[i];
        a[i] = temp;
```

```

        heapify(a, i, 0);
    }
}

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int a[n];

    srand(time(NULL));

    for(int i = 0; i < n; i++)
        a[i] = rand() % 10000;

    clock_t start, end;

    start = clock();

    for(int i=0;i<1000;i++)
    {
        heapSort(a,n-1);
    }

    end = clock();

    double time_taken =
        ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\nFirst 20 Sorted Elements:\n");

    for(int i = 0; i < n && i < 20; i++)
        printf("%d ", a[i]);

    printf("\n\nTime Taken = %.3f milliseconds\n",
        time_taken * 1000);

    return 0;
}

```

OUTPUT:

```
Enter number of elements: 100

First 20 Sorted Elements:
19 31 212 283 338 395 407 444 468 500 503 514 659 898 942 1313 1362 1382 1491 1655

Time Taken = 4.000 milliseconds

Process returned 0 (0x0)   execution time : 9.907 s
Press any key to continue.
```

```
Enter number of elements: 500

First 20 Sorted Elements:
25 47 60 66 94 95 97 101 106 130 139 167 191 236 263 266 320 338 364 367

Time Taken = 35.000 milliseconds

Process returned 0 (0x0)   execution time : 8.443 s
Press any key to continue.
|
```

```
Enter number of elements: 1000

First 20 Sorted Elements:
6 17 50 54 66 78 85 92 101 123 129 129 147 148 153 162 163 166 176 180

Time Taken = 87.000 milliseconds

Process returned 0 (0x0)   execution time : 6.634 s
Press any key to continue.
|
```

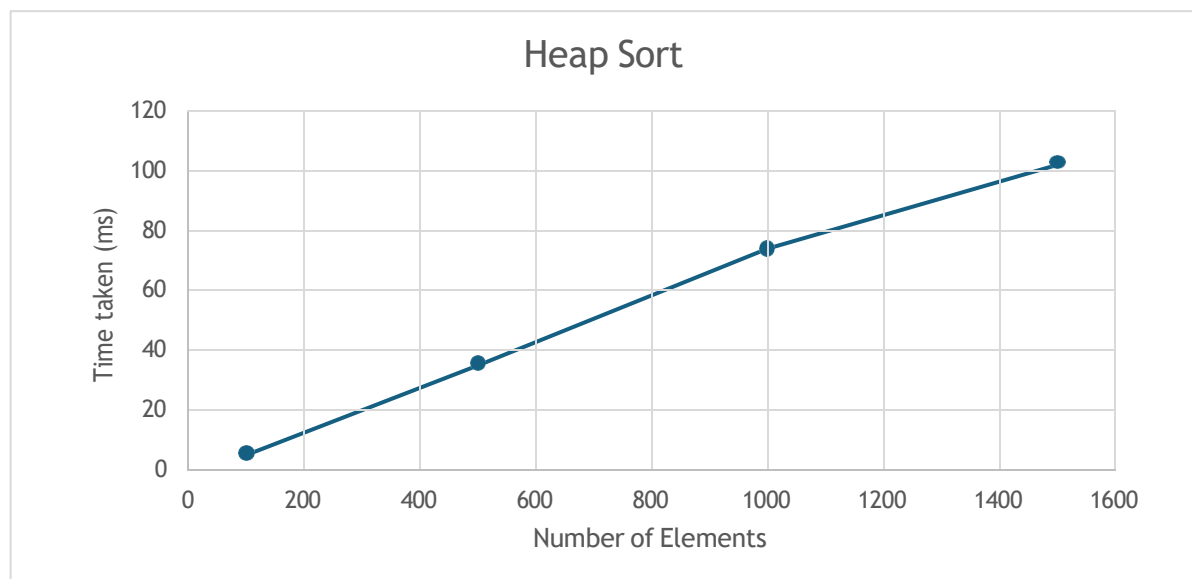
```
Enter number of elements: 1500

First 20 Sorted Elements:
1 3 5 11 15 20 23 36 37 55 55 58 81 100 116 118 126 133 134 137

Time Taken = 167.000 milliseconds

Process returned 0 (0x0)   execution time : 7.791 s
Press any key to continue.
|
```

GRAPH :



Lab program 6

Implement 0/1 Knapsack problem using dynamic programming.

C code

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W, i, w;

    printf("Enter number of items: ");
    scanf("%d", &n);

    int wt[n], val[n];

    printf("Enter weights of items:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }

    printf("Enter values of items:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }

    printf("Enter capacity of knapsack: ");
    scanf("%d", &W);

    int dp[n+1][W+1];

    for(i = 0; i <= n; i++) {
        for(w = 0; w <= W; w++) {
            if(i == 0 || w == 0)
                dp[i][w] = 0;
            else if(wt[i-1] <= w)
```

```

        dp[i][w] = max(val[i-1] + dp[i-1][w - wt[i-1]],
                        dp[i-1][w]);
    else
        dp[i][w] = dp[i-1][w];
    }
}

printf("\nMaximum value = %d\n", dp[n][W]);

return 0;
}

```

Output

Case 1

```

"C:\Users\ramit\OneDrive\De:  ×  +  ∨
Enter number of items: 3
Enter weights of items:
10 20 30
Enter values of items:
60 100 120
Enter capacity of knapsack: 50

Maximum value = 220

```

Case 2

```

"C:\Users\ramit\OneDrive\De:  ×  +  ∨
Enter number of items: 3
Enter weights of items:
10 20 30
Enter values of items:
60 100 120
Enter capacity of knapsack: 5

Maximum value = 0

```

Case 3

```

"C:\Users\ramit\OneDrive\De:  ×  +  ∨
Enter number of items: 3
Enter weights of items:
5 10 15
Enter values of items:
10 20 30
Enter capacity of knapsack: 30

Maximum value = 60

```

Lab Program 7

Implement All Pair Shortest paths problem using Floyd's algorithm.

C Code

```
#include <stdio.h>
#define INF 9999

void floydWarshall(int n, int graph[10][10]) {
    int dist[10][10];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            dist[i][j] = graph[i][j];
        }
    }

    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    printf("\nShortest distance matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (dist[i][j] == INF)
                printf("%7s", "INF");
            else
                printf("%7d", dist[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int n;
    int graph[10][10];
    printf("Enter the number of vertices (max 10): ")
```



```
scanf("%d", &n);

printf("Enter the adjacency matrix (use %d for INF):\n", INF);
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        scanf("%d", &graph[i][j]);
    }
}

floydWarshall(n, graph);

return 0;
}
```

Output

Case 1

```
"C:\Users\ramit\OneDrive\De:  ×  +  v
Enter the number of vertices (max 10): 4
Enter the adjacency matrix (use 9999 for INF):
0 5 9999 10
9999 0 3 9999
9999 9999 0 1
9999 9999 9999 0

Shortest distance matrix:
    0      5      8      9
  INF      0      3      4
  INF     INF      0      1
  INF     INF     INF      0
```

Case 2

```
"C:\Users\ramit\OneDrive\De:  ×  +  v
Enter the number of vertices (max 10): 3
Enter the adjacency matrix (use 9999 for INF):
0 2 4
2 0 1
4 1 0

Shortest distance matrix:
    0      2      3
    2      0      1
    3      1      0
```

Case 3

```
"C:\Users\ramit\OneDrive\De:  ×  +  v
Enter the number of vertices (max 10): 4
Enter the adjacency matrix (use 9999 for INF):
0 4 9999 9999
9999 0 2 9999
9999 9999 0 9999
9999 9999 9999 0

Shortest distance matrix:
    0      4      6  INF
  INF      0      2  INF
  INF     INF      0  INF
  INF     INF     INF      0
```

Lab Program 8a

Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

C Code

```
#include <stdio.h>
#include <limits.h>

int minKey(int key[], int mstSet[], int n)
{
    int min = INT_MAX, min_index = -1;

    for(int v = 0; v < n; v++)
    {
        if(!mstSet[v] && key[v] < min)
        {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}

void primMST(int n, int graph[n][n])
{
    int parent[n];
    int key[n];
    int mstSet[n];

    for(int i = 0; i < n; i++)
    {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for(int count = 0; count < n - 1; count++)
    {
        int u = minKey(key, mstSet, n);
        mstSet[u] = 1;
```

```

        for(int v = 0; v < n; v++)
        {
            if(graph[u][v] &&
               !mstSet[v] &&
               graph[u][v] < key[v])
            {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }

    printf("\nEdge\tWeight\n");
    for(int i = 1; i < n; i++)
    {
        printf("%d - %d\t%d\n",
               parent[i], i,
               graph[i][parent[i]]);
    }
}

int main()
{
    int n;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    int graph[n][n];

    printf("Enter adjacency matrix:\n");

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }

    primMST(n, graph);

    return 0;
}

```

Output

Case 1

```
"C:\Users\ramit\OneDrive\De: X + v
Enter number of vertices: 4
Enter adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 5
6 8 5 0

Edge    Weight
0 - 1    2
1 - 2    3
2 - 3    5
```

Case 2

```
"C:\Users\ramit\OneDrive\De: X + v
Enter number of vertices: 5
Enter adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edge    Weight
0 - 1    2
1 - 2    3
0 - 3    6
1 - 4    5
```

Case 3

```
"C:\Users\ramit\OneDrive\De: X + v
Enter number of vertices: 6
Enter adjacency matrix:
0 3 1 0 0 0
3 0 7 5 1 0
1 7 0 2 0 4
0 5 2 0 6 3
0 1 0 6 0 8
0 0 4 3 8 0

Edge    Weight
0 - 1    3
0 - 2    1
2 - 3    2
1 - 4    1
3 - 5    3
```

Lab Program 8b

Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm

C Code

```
#include <stdio.h>

#define MAX 30

struct Edge
{
    int u, v, w;
};

int find(int parent[], int i)
{
    if(parent[i] != i)
        parent[i] = find(parent, parent[i]);
    return parent[i];
}

void unionSets(int parent[], int rank[], int x, int y)
{
    int xroot = find(parent, x);
    int yroot = find(parent, y);

    if(rank[xroot] < rank[yroot])
        parent[xroot] = yroot;
    else if(rank[xroot] > rank[yroot])
        parent[yroot] = xroot;
    else
    {
        parent[yroot] = xroot;
        rank[xroot]++;
    }
}

void kruskal(struct Edge edges[], int V, int E)
{
    int parent[MAX], rank[MAX];
    struct Edge result[MAX];

    int e = 0, i = 0;
```

```

for(int v = 0; v < V; v++)
{
    parent[v] = v;
    rank[v] = 0;
}

for(int a = 0; a < E - 1; a++)
{
    for(int b = 0; b < E - a - 1; b++)
    {
        if(edges[b].w > edges[b + 1].w)
        {
            struct Edge temp = edges[b];
            edges[b] = edges[b + 1];
            edges[b + 1] = temp;
        }
    }
}

while(e < V - 1 && i < E)
{
    struct Edge next = edges[i++];

    int x = find(parent, next.u);
    int y = find(parent, next.v);

    if(x != y)
    {
        result[e++] = next;
        unionSets(parent, rank, x, y);
    }
}

printf("\nEdges in the Minimum Cost Spanning Tree:\n");

int minCost = 0;

for(int j = 0; j < e; j++)
{
    printf("%d -- %d == %d\n",
        result[j].u,
        result[j].v,
        result[j].w);
}

```

```

        minCost += result[j].w;
    }

    printf("Minimum Cost of Spanning Tree: %d\n", minCost);
}

int main()
{
    int V, E;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter number of edges: ");
    scanf("%d", &E);

    struct Edge edges[E];

    printf("Enter source destination weight for each edge:\n");

    for(int i = 0; i < E; i++)
    {
        scanf("%d %d %d",
            &edges[i].u,
            &edges[i].v,
            &edges[i].w);
    }

    kruskal(edges, V, E);

    return 0;
}

```

Output

Case 1

```
"C:\Users\ramit\OneDrive\De: X + v
Enter number of vertices: 4
Enter number of edges: 5
Enter source destination weight for each edge:
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Edges in the Minimum Cost Spanning Tree:
2 -- 3 == 4
0 -- 3 == 5
0 -- 1 == 10
Minimum Cost of Spanning Tree: 19
```

Case 2

```
"C:\Users\ramit\OneDrive\De: X + v
Enter number of vertices: 5
Enter number of edges: 7
Enter source destination weight for each edge:
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 9

Edges in the Minimum Cost Spanning Tree:
0 -- 1 == 2
1 -- 2 == 3
1 -- 4 == 5
0 -- 3 == 6
Minimum Cost of Spanning Tree: 16
```


Lab Program 9

Implement Fractional Knapsack using Greedy technique.

C Code

```
#include <stdio.h>

struct Item {
    int profit, weight;
    float ratio;
};

void sort(struct Item items[], int n) {
    int i, j;
    struct Item temp;

    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(items[i].ratio < items[j].ratio) {
                temp = items[i];
                items[i] = items[j];
                items[j] = temp;
            }
        }
    }
}

float fractionalKnapsack(struct Item items[], int n, int capacity) {
    float totalProfit = 0.0;
    int i;

    for(i = 0; i < n; i++) {
        if(items[i].weight <= capacity) {
            totalProfit += items[i].profit;
            capacity -= items[i].weight;
        } else {
            totalProfit += items[i].ratio * capacity;
            break;
        }
    }

    return totalProfit;
}
```

```
int main() {
    int n, i, capacity;
    struct Item items[50];

    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter profit and weight of each item:\n");
    for(i = 0; i < n; i++) {
        scanf("%d %d", &items[i].profit, &items[i].weight);
        items[i].ratio = (float)items[i].profit / items[i].weight;
    }

    printf("Enter capacity of knapsack: ");
    scanf("%d", &capacity);

    sort(items, n);

    float maxProfit = fractionalKnapsack(items, n, capacity);

    printf("Maximum Profit = %.2f\n", maxProfit);

    return 0;
}
```

Output

Case 1

```
"C:\Users\ramit\OneDrive\De: X + v
Enter number of items: 3
Enter profit and weight of each item:
60 10
100 20
120 30
Enter capacity of knapsack: 50
Maximum Profit = 240.00
```

Case 2

```
"C:\Users\ramit\OneDrive\De: X + v
Enter number of items: 3
Enter profit and weight of each item:
10 5
20 10
30 15
Enter capacity of knapsack: 30
Maximum Profit = 60.00
```

Case 3

```
"C:\Users\ramit\OneDrive\De: X + v
Enter number of items: 3
Enter profit and weight of each item:
60 10
100 20
120 30
Enter capacity of knapsack: 5
Maximum Profit = 30.00
```

Lab Program 10

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

C Code

```
#include<stdio.h>
#define inf 9999
#define max 20
int main()
{
    int i,j,k,u,v,n;
    int c[max][max],d[max],vis[max];
    printf("Enter the number of vertices:");
    scanf("%d",&n);
    printf("\nEnter the cost matrix:");
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
            scanf("%d",&c[i][j]);
    }
    printf("\nEnter the source vertex :");
    scanf("%d",&v);
    for(i=0;i<n;i++)
    {
        d[i]=c[v][i];
        vis[i]=0;
    }
    d[v]=0;
    vis[v]=1;
    for(k = 1; k < n; k++) {
        int min = inf;
        u = -1;

        for(i = 0; i < n; i++) {
            if(!vis[i] && d[i] < min) {
                min = d[i];
                u = i;
            }
        }

        if(u == -1) break;

        vis[u] = 1;
```

```
    for(i = 0; i < n; i++) {  
        if(!vis[i] && c[u][i] != inf &&  
            d[i] > d[u] + c[u][i]) {  
            d[i] = d[u] + c[u][i];  
        }  
    }  
}  
  
printf("\nShortest distances from %d:\n", v);  
for(i = 0; i < n; i++) {  
    printf("%d -> %d = %d\n", v, i, d[i]);  
}  
  
return 0;  
}
```

Output

Case 1

```
"C:\Users\ramit\OneDrive\De: X + v
Enter the number of vertices:4

Enter the cost matrix:
0 5 9999 10
9999 0 3 9999
9999 9999 0 1
9999 9999 9999 0

Enter the source vertex :0

Shortest distances from 0:
0 -> 0 = 0
0 -> 1 = 5
0 -> 2 = 8
0 -> 3 = 9
```

Case 2

```
"C:\Users\ramit\OneDrive\De: X + v
Enter the number of vertices:5

Enter the cost matrix:
0 2 4 7 9
2 0 1 5 6
4 1 0 2 3
7 5 2 0 4
9 6 3 4 0

Enter the source vertex :1

Shortest distances from 1:
1 -> 0 = 2
1 -> 1 = 0
1 -> 2 = 1
1 -> 3 = 3
1 -> 4 = 4
```

Case 3

```
"C:\Users\ramit\OneDrive\De: X + v
Enter the number of vertices:5

Enter the cost matrix:
0 4 9999 9999 9999
9999 0 2 9999 9999
9999 9999 0 9999 9999
9999 9999 9999 0 3
9999 9999 9999 9999 0

Enter the source vertex :0

Shortest distances from 0:
0 -> 0 = 0
0 -> 1 = 4
0 -> 2 = 6
0 -> 3 = 9999
0 -> 4 = 9999
```

Lab Program 11

Implement “N-Queens Problem” using Backtracking.

C Code

```
#include <stdio.h>

int board[20], n;

int safe(int row, int col){
    for(int i=0;i<row;i++){
        if(board[i]==col ||
           board[i]-i == col-row ||
           board[i]+i == col+row)
            return 0;
    }
    return 1;
}

void solve(int row){
    if(row==n){
        for(int i=0;i<n;i++){
            for(int j=0;j<n;j++){
                printf(board[i]==j ? "Q " : ". ");
            }
            printf("\n");
        }
        printf("\n");
        return;
    }

    for(int col=0;col<n;col++){
        if(safe(row,col)){
            board[row]=col;
            solve(row+1);
        }
    }
}
```

```
int main(){
    printf("Enter the value of n:");
    scanf("%d",&n);
    solve(0);
    return 0;
}
```

Output

Case 1



```
"C:\Users\ramit\OneDrive\De" X + v
Enter the value of n:1
Q
```

Case 2



```
"C:\Users\ramit\OneDrive\De" X + v
Enter the value of n:4
. Q . .
. . . Q
Q . . .
. . Q .

. . Q .
Q . . .
. . . Q
. Q . .
```


Case 3

[illegible]

26. Remove Duplicates from Sorted Array

Given an integer array `nums` sorted in non-decreasing order, remove the duplicates **in-place** such that each unique element appears only once. The relative order of the elements should be kept the same.

Consider the number of *unique elements* in `nums` to be `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the unique numbers in sorted order. The remaining elements beyond index `k - 1` can be ignored.

C Code

```
int removeDuplicates(int* nums, int numsSize) {  
    if (numsSize == 0) return 0;  
  
    int k = 1;  
  
    for (int i = 1; i < numsSize; i++) {  
        if (nums[i] != nums[i - 1]) {  
            nums[k++] = nums[i];  
        }  
    }  
  
    return k;  
}
```

Output

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums =
[1,1,2]

Output

[1,2]

Accepted 362 / 362 testcases passed

 Arya Piyush submitted at Jun 10, 2026 02:33

 Analysis

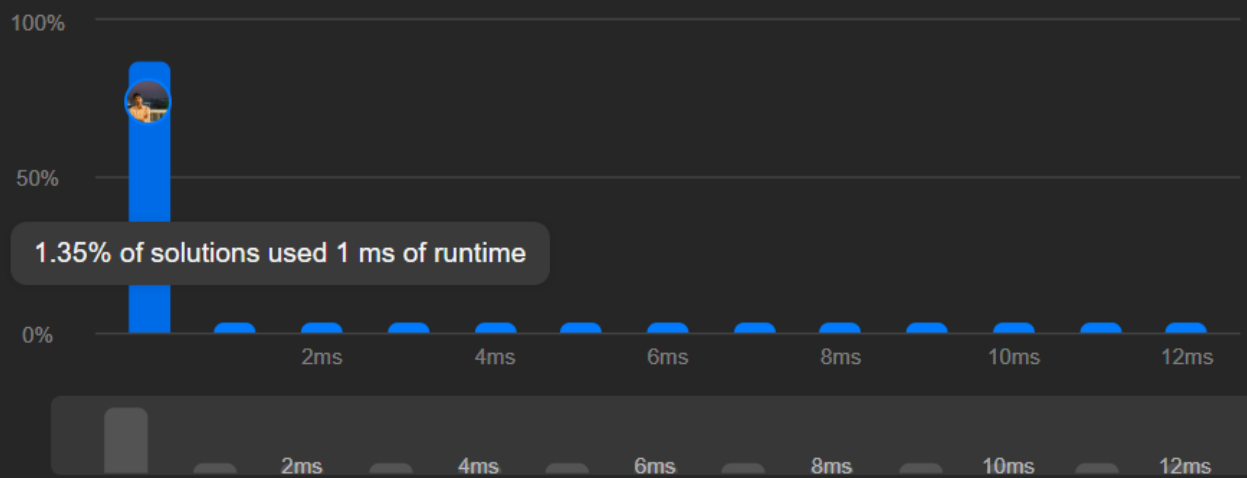
 Solution

 Runtime

0 ms | Beats 100.00% 

 Memory

12.92 MB | Beats 39.10%



21. Merge Two Sorted Lists

You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.

Return *the head of the merged linked list*.

C code :

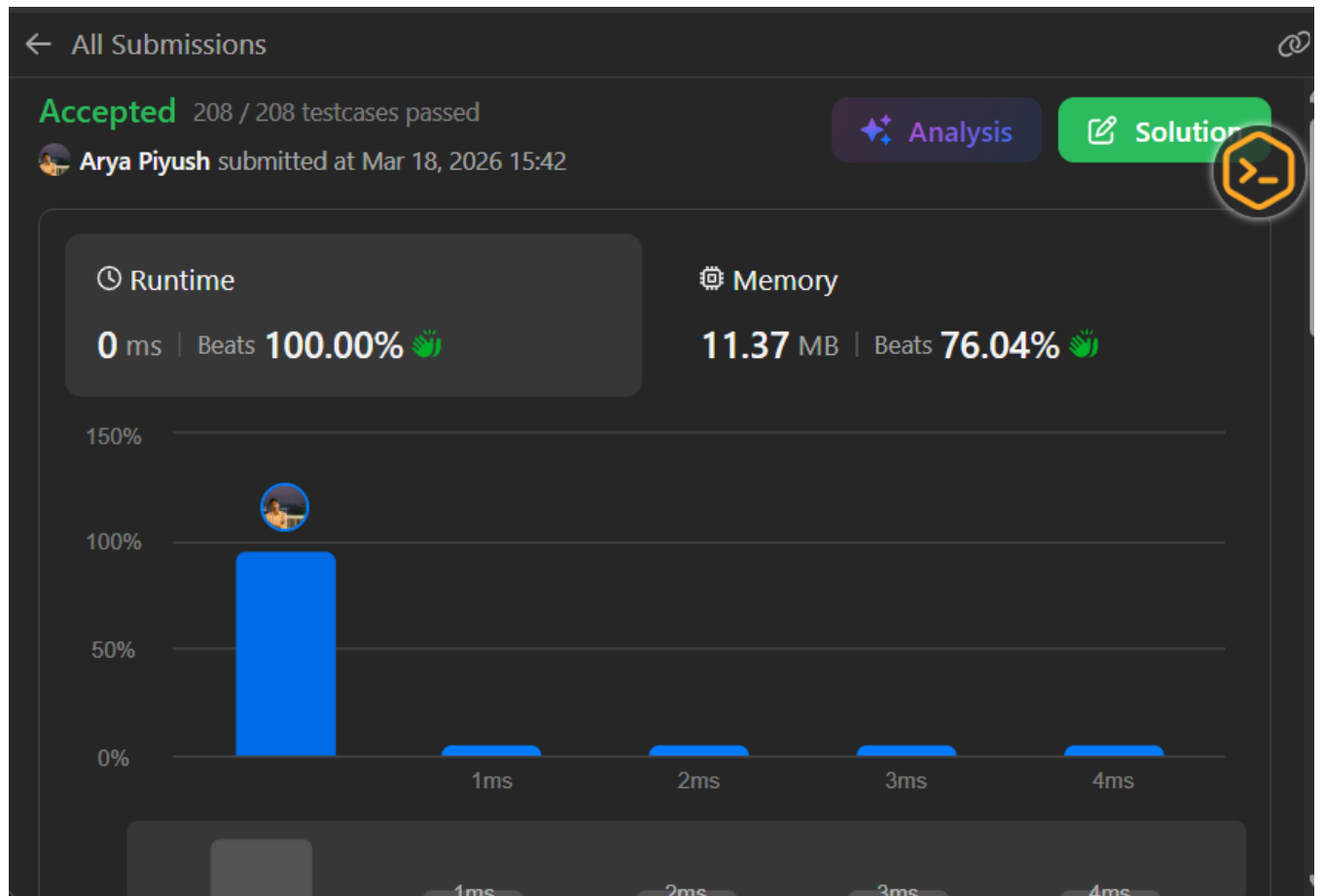
```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     struct ListNode *next;
 * };
 */
struct ListNode* mergeTwoLists(struct ListNode* list1, struct ListNode* list2) {
    struct ListNode dummy;
    struct ListNode* current = &dummy;
    dummy.next = NULL;

    while (list1 != NULL && list2 != NULL) {
        if (list1->val < list2->val) {
            current->next = list1;
            list1 = list1->next;
        } else {
            current->next = list2;
            list2 = list2->next;
        }
        current = current->next;
    }

    if (list1 != NULL) {
        current->next = list1;
    } else {
        current->next = list2;
    }

    return dummy.next;
}
```

Output :



1928. Minimum Cost to Reach Destination in Time

There is a country of n cities numbered from 0 to $n - 1$ where all the cities are connected by bi-directional roads. The roads are represented as a 2D integer array `edges` where `edges[i] = [xi, yi, timei]` denotes a road between cities x_i and y_i that takes $time_i$ minutes to travel. There may be multiple roads of differing travel times connecting the same two cities, but no road connects a city to itself.

Each time you pass through a city, you must pay a passing fee. This is represented as a 0-indexed integer array `passingFees` of length n where `passingFees[j]` is the amount of dollars you must pay when you pass through city j .

In the beginning, you are at city 0 and want to reach city $n - 1$ in `maxTime` minutes or less. The cost of your journey is the summation of passing fees for each city that you passed through at some moment of your journey (including the source and destination cities).

Given `maxTime`, `edges`, and `passingFees`, return *the minimum cost to complete your journey, or -1 if you cannot complete it within maxTime minutes.*

C Code :

```
#include <stdio.h>

#include <stdlib.h>

#include <limits.h>

#define MAXN 1001

#define MAXT 1001

typedef struct {

    int cost;

    int time;

    int node;

} State;

typedef struct {

    State *arr;

    int size;

} 57 | Page
```

```
} MinHeap;
```

```
void swap(State *a, State *b) {
```

```
    State temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
void push(MinHeap *heap, State val) {
```

```
    int i = heap->size++;
```

```
    heap->arr[i] = val;
```

```
    while (i > 0) {
```

```
        int p = (i - 1) / 2;
```

```
        if (heap->arr[p].cost <= heap->arr[i].cost) break;
```

```
        swap(&heap->arr[p], &heap->arr[i]);
```

```
        i = p;
```

```
    }
```

```
}
```

```
State pop(MinHeap *heap) {
```

```
    State top = heap->arr[0];
```

```
    heap->arr[0] = heap->arr[--heap->size];
```

```
    int i = 0;
```

```
    while (1) {
```

```
        int l = 2*i + 1, r = 2*i + 2, smallest = i;
```

```
        if (l < heap->size && heap->arr[l].cost < heap->arr[smallest].cost)
```



```

        smallest = l;

        if (r < heap->size && heap->arr[r].cost < heap->arr[smallest].cost)

            smallest = r;

        if (smallest == i) break;

        swap(&heap->arr[i], &heap->arr[smallest]);

        i = smallest;
    }

    return top;
}

```

```

int minCost(int maxTime, int** edges, int edgesSize, int* edgesColSize,

            int* passingFees, int n) {

    // Build adjacency list

    int adj[MAXN][MAXN];

    int timeCost[MAXN][MAXN];

    int deg[MAXN] = {0};

    for (int i = 0; i < edgesSize; i++) {

        int u = edges[i][0], v = edges[i][1], t = edges[i][2];

        adj[u][deg[u]] = v;

        timeCost[u][deg[u]++] = t;

        adj[v][deg[v]] = u;

        timeCost[v][deg[v]++] = t;

    }
}

```

```

// dp[node][time] = min cost

static int dp[MAXN][MAXT];

for (int i = 0; i < n; i++)

    for (int t = 0; t <= maxTime; t++)

        dp[i][t] = INT_MAX;


MinHeap heap;

heap.arr = malloc(sizeof(State) * 100000);

heap.size = 0;


dp[0][0] = passingFees[0];

push(&heap, (State){passingFees[0], 0, 0});


while (heap.size > 0) {

    State cur = pop(&heap);

    int cost = cur.cost, time = cur.time, node = cur.node;


    if (node == n - 1) return cost;


    if (dp[node][time] < cost) continue;


    for (int i = 0; i < deg[node]; i++) {

        int nei = adj[node][i];

        int t = timeCost[node][i];

        int newTime = time + t;

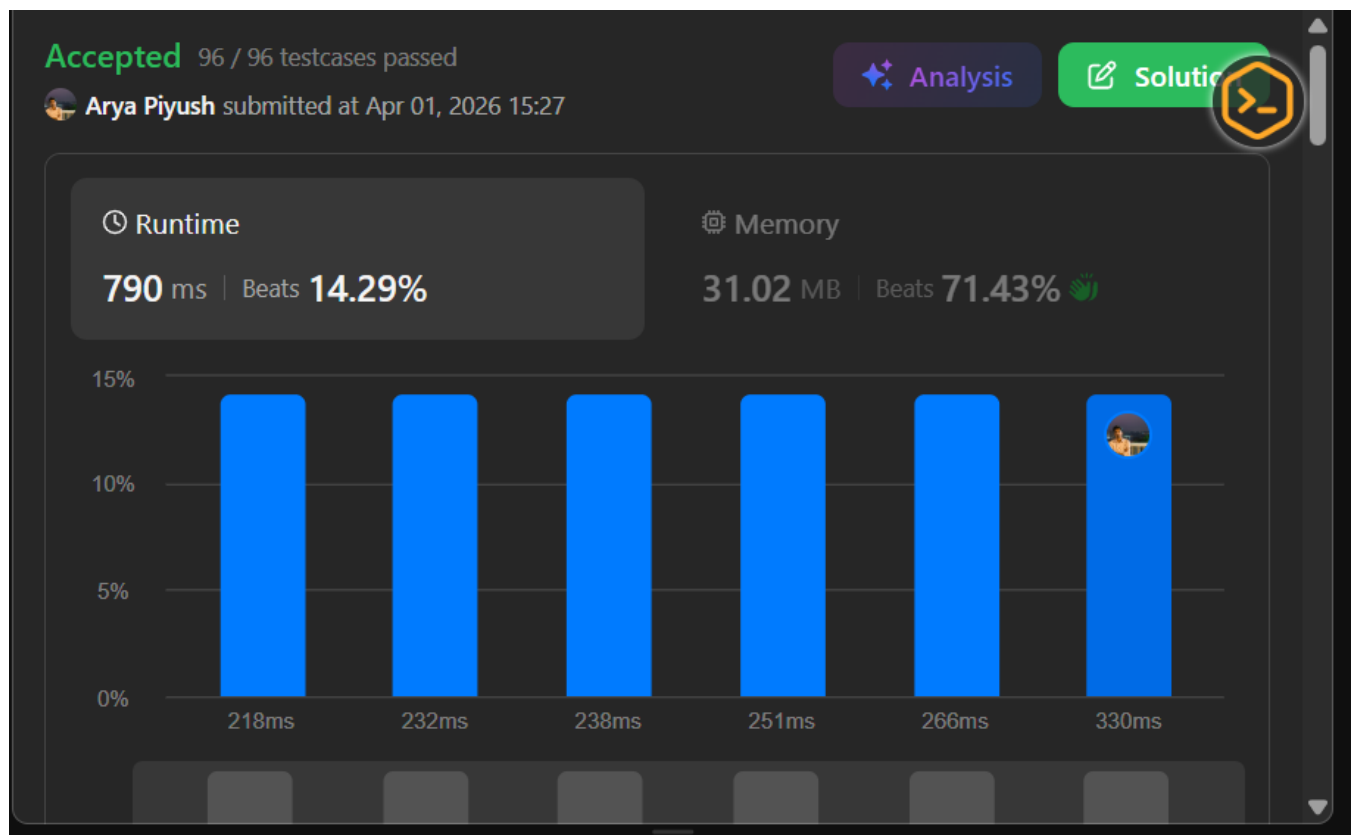

        if (newTime > maxTime) continue;


        int newCost = cost + passingFees[nei];

```

```
        if (dp[nei][newTime] > newCost) {  
            dp[nei][newTime] = newCost;  
            push(&heap, (State){newCost, newTime, nei});  
        }  
    }  
}  
  
return -1;  
}
```

output :



322. Coin Change

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return *the fewest number of coins that you need to make up that amount*. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

C CODE:

```
int coinChange(int* coins, int coinsSize, int amount) {
    int dp[amount + 1];

    // Initialize DP array
    for (int i = 0; i <= amount; i++) {
        dp[i] = amount + 1; // acts like infinity
    }

    dp[0] = 0;

    // Build up the dp array
    for (int i = 1; i <= amount; i++) {
        for (int j = 0; j < coinsSize; j++) {
            if (i - coins[j] >= 0) {
                int sub = dp[i - coins[j]] + 1;
                if (sub < dp[i]) {
                    dp[i] = sub;
                }
            }
        }
    }

    return (dp[amount] > amount) ? -1 : dp[amount];
}
```

OUTPUT :

Accepted 189 / 189 testcases passed

Arya Piyush submitted at Apr 15, 2026 15:28

Analysis

Solution

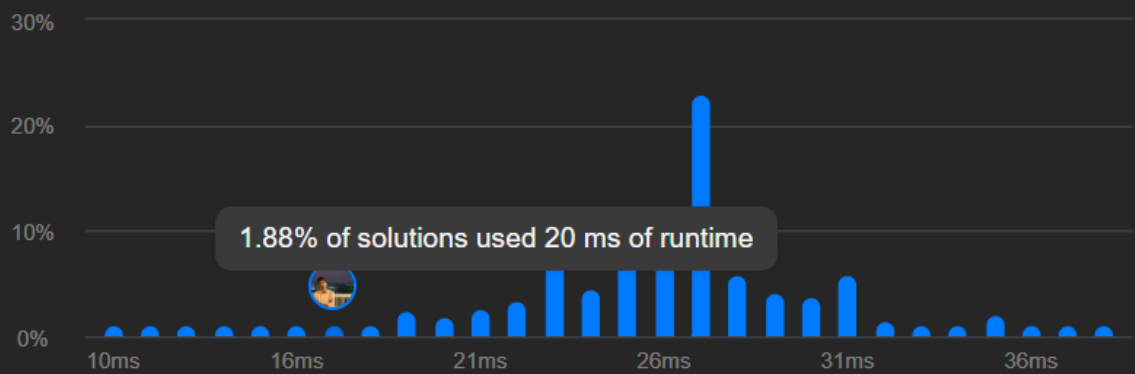


Runtime

17 ms | Beats 98.93%

Memory

8.87 MB | Beats 68.93%



1334. Find the City With the Smallest Number of Neighbors at a Threshold Distance

There are n cities numbered from 0 to $n-1$. Given the array `edges` where `edges[i] = [from $i$ , to $i$ , weight $i$ ]` represents a bidirectional and weighted edge between cities `from $i$` and `to $i$` , and given the integer `distanceThreshold`.

Return the city with the smallest number of cities that are reachable through some path and whose distance is at most `distanceThreshold`. If there are multiple such cities, return the city with the greatest number.

Notice that the distance of a path connecting cities i and j is equal to the sum of the edges' weights along that path.

C CODE:

```
#include <limits.h>
```

```
#define INF 1000000000
```

```
#define MAXN 101
```

```
int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int
distanceThreshold) {
    int dist[MAXN][MAXN];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j)
                dist[i][j] = 0;
            else
                dist[i][j] = INF;
        }
    }
    for (int i = 0; i < edgesSize; i++) {
        int u = edges[i][0];
        int v = edges[i][1];
        int w = edges[i][2];
        dist[u][v] = w;
        dist[v][u] = w;
    }
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}
```

```

int minCount = INT_MAX;
int resultCity = -1;
for (int i = 0; i < n; i++) {
    int count = 0;
    for (int j = 0; j < n; j++) {
        if (i != j && dist[i][j] <= distanceThreshold) {
            count++;
        }
    }
    if (count <= minCount) {
        minCount = count;
        resultCity = i;
    }
}
return resultCity;
}

```

OUTPUT:



638. Shopping Offers

In LeetCode Store, there are n items to sell. Each item has a price. However, there are some special offers, and a special offer consists of one or more different kinds of items with a sale price.

You are given an integer array `price` where `price[i]` is the price of the i^{th} item, and an integer array `needs` where `needs[i]` is the number of pieces of the i^{th} item you want to buy.

You are also given an array `special` where `special[i]` is of size $n + 1$ where `special[i][j]` is the number of pieces of the j^{th} item in the i^{th} offer and `special[i][n]` (i.e., the last integer in the array) is the price of the i^{th} offer.

Return *the lowest price you have to pay for exactly certain items as given, where you could make optimal use of the special offers*. You are not allowed to buy more items than you want, even if that would lower the overall price. You could use any of the special offers as many times as you want.

C CODE:

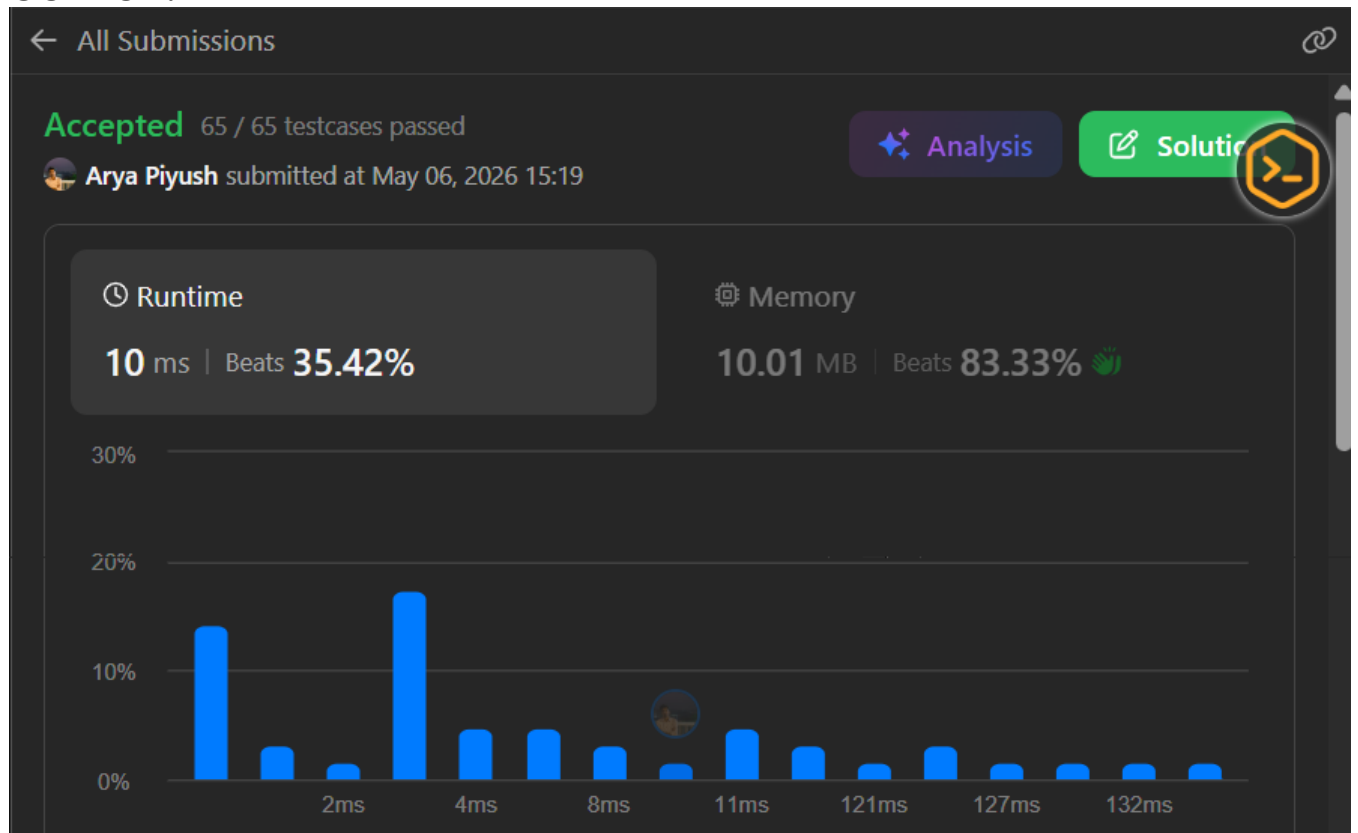
```
int dfs(int* price,int n,int** sp,int m,int* needs){
    int res=0;
    for(int i=0;i<n;i++) res+=needs[i]*price[i];

    for(int i=0;i<m;i++){
        int ok=1, sum=0;
        for(int j=0;j<n;j++){
            if(needs[j]<sp[i][j]){ok=0;break;}
            sum += sp[i][j]*price[j]; // individual cost
        }
        if(!ok || sum <= sp[i][n]) continue;

        for(int j=0;j<n;j++) needs[j]-=sp[i][j];
        int val=dfs(price,n,sp,m,needs)+sp[i][n];
        if(val<res) res=val;
        for(int j=0;j<n;j++) needs[j]+=sp[i][j];
    }
    return res;
}

int shoppingOffers(int* price, int priceSize, int** special, int specialSize,
    int* specialColSize, int* needs, int needsSize) {
    return dfs(price,priceSize,special,specialSize,needs);
}
```

OUTPUT:



210. Course Schedule II

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you must take course b_i first if you want to take course a_i.

- For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return *the ordering of courses you should take to finish all courses*. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

C CODE:

```
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
// LeetCode 210 - Course Schedule II
// Topological Sort using BFS (Kahn's Algorithm)

int* findOrder(int n, int** pre, int m, int* col, int* returnSize) {
    int *indeg = calloc(n, sizeof(int));
    int **g = calloc(n, sizeof(int*));
    int *sz = calloc(n, sizeof(int));

    for(int i = 0; i < m; i++) {
        int a = pre[i][0], b = pre[i][1];
        g[b] = realloc(g[b], (sz[b] + 1) * sizeof(int));
        g[b][sz[b]++] = a;
        indeg[a]++;
    }

    int *q = malloc(n * sizeof(int)), f = 0, r = 0;
    for(int i = 0; i < n; i++)
        if(indeg[i] == 0) q[r++] = i;

    int *ans = malloc(n * sizeof(int)), k = 0;

    while(f < r) {
        int u = q[f++];
        ans[k++] = u;

        for(int i = 0; i < sz[u]; i++) {
            int v = g[u][i];
            if(--indeg[v] == 0) q[r++] = v;
        }
    }
}
```

```
    }  
  
    *returnSize = (k == n) ? n : 0;  
    return (k == n) ? ans : NULL;  
}
```

OUTPUT:

